College of Engineering, Construction and Living Sciences
Bachelor of Information Technology
ID607001: Introductory Application Development Concepts
Level 6, Credits 15

Project

Assessment Overview

In this **individual** assessment, you will develop two web applications using **Express** and **SvelteKit**.

Learning Outcome

At the successful completion of this course, learners will be able to:

1. Design and build secure applications with dynamic database functionality following an appropriate software development methodology.

Assessments

Assessment	Weighting	Due Date	Learning Outcome
Practical	20%	Sunday 7th September at 11.59 PM	1
Project	80%	Sunday 9th November at 11.59 PM	1

Conditions of Assessment

You will complete this assessment mostly during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment will need to be completed by **Sunday 9th November** at **11.59 PM**.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of **50%** across all assessments in **ID607001: Introductory Application Development Concepts**.

Submission

You **must** submit all application files via **GitHub Classroom**. Here is the URL to the repository you will use for your submission – https://classroom.github.com/a/8sCyquQ_. If you do not have not one, create a **.gitignore** and add the ignored files in this resource - https://raw.githubusercontent.com/github/gitignore/main/Node.gitignore.

Create a branch called **project**. The latest application files in the **project** branch will be used to mark against the marking rubric. Please test before you submit. Partial marks **will not** be given for incomplete functionality. Late submissions will incur a **10% penalty per day**, rolling over at **12.00 AM**.

Authenticity

All parts of your submitted assessment **must** be completely your work. Do your best to complete this assessment without using AI tools. You need to demonstrate to the course lecturer that you can meet the learning outcome for this assessment.

Learning to use AI tools is an important skill. While AI tools are powerful, you **must** be aware of the following:

- If you provide an AI tool with a prompt that is not refined enough, it may generate a not-so-useful response
- Do not trust the AI tool's responses blindly. You **must** still use your judgement and may need to do additional research to determine if the response is correct
- Acknowledge what AI tool you have used. In the assessment's repository **README.md** file, please include what prompt(s) you provided to the AI tool and how you used the response(s) to help you with your work

It also applies to code snippets retrieved from **StackOverflow** and **GitHub**.

Failure to do this may result in a mark of **zero** for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with **Otago Polytechnic** policies. Learners can view policies on the **Otago Polytechnic** website located at https://www.op.ac.nz/about-us/governance-and-management/policies.

Extensions

Familiarise yourself with the assessment due date. Extensions will **only** be granted if you are unable to complete the assessment by the due date because of **unforeseen circumstances outside your control**. The length of

the extension granted will depend on the circumstances and **must** be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted on the due date and for poor time management or pressure of other assessments.

Resits

Resits and reassessments are not applicable in **ID607001: Introductory Application Development Concepts**.

Instructions

Functionality - Learning Outcome 1 (25%)

- Backend application with the following functionality:
  - Five **models** with a minimum of four **fields**.
  - One model should have an enum field.
  - One **one-to-many** relationship and one **many-to-many** relationship.
  - **CRUD** operations for each model.
  - Validation on **Create** and **Update** operations.
  - **Filter**, **sort** and **paginate** using **query parameters**.
  - Allow users to register, login and logout.
  - Role-based access control:
    - An **admin user** can perform **CRUD** operations on all **models**.
    - A **normal user** can perform **Read all** and **Read by UUID** operations on all **models**.
  - Content negotiation to support **JSON** format.
  - Seed script to populate the database with initial data.
  - API tests covering **CRUD** operations for each model.
  - Deploy to **Render**.
- Frontend application with the following functionality:
  - Consume the backend application on **Render** to perform **CRUD** operations based on role.
  - Styled using **Bootstrap**.
  - Authentication with secure token handling.
  - Display data with support for **filtering**, **sorting** and **paging**.
  - Forms with validation for **Create** and **Update** operations.
  - Render UI elements and actions based on role.
  - Ten end-to-end tests covering the main features.
  - Deploy to **Vercel**.

Code Quality and Best Practices - Learning Outcome 1 (40%)

- Write clean, readable and maintainable code.
- Use modular, reusable components and avoid code duplication.
- Keep code simple and easy to understand.
- Optimise performance by managing resources efficiently.

Version Control - Learning Outcome 1 (5%)

- Regular commits are made throughout development, demonstrating meaningful progress.
- Commit messages are clear and descriptive.
- The repository shows a clean and logical commit history that reflects the development process of the applications.

Reflection - Learning Outcome 1 (5%)

- Write a short reflection (300-500 words) on your development process and learning throughout the project, including:
  - Challenges you faced and how you overcame them.
  - What you would do differently in future projects.
  - New skills, tools or practices you learned during the project.

Presentation - Learning Outcome 1 (5%)

- Record a 5-10 minute screen recording of your applications.
- Demonstrate the backend and frontend features.
- Make sure the video is clear and easy to follow.
- Submit a link to the presentation in your repository's **README.md** file.

Marking Rubric

Functionality - Learning Outcome 1 (25%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Models and Relationships (4%)	All five models implemented with 4+ fields each. Enum field present. Both one-to-many and many-to-many relationships correctly implemented.	Five models with 4+ fields. Minor issues with enum or one relationship type.	4-5 models present. Some fields or relationships missing or incorrect.	Fewer than 4 models or significant issues with structure.
CRUD Operations and Validation (4%)	Complete CRUD for all models with robust validation on Create and Update. Error handling implemented.	CRUD operations present for all models. Validation mostly complete with minor gaps.	CRUD operations present but some missing or incomplete. Basic validation implemented.	Multiple CRUD operations missing or not functional. Little to no validation.
Filter, Sort and Pagination (3%)	Filter, sort and pagination fully functional using query parameters across all models.	Filter, sort and pagination implemented but with minor issues or limited to some models.	Basic filtering, sorting or pagination present but incomplete implementation.	Filtering, sorting and pagination missing or non-functional.
Authentication and Authorization (3%)	Complete registration, login, logout. Role-based access control fully implemented with admin and normal user roles enforced correctly.	Authentication functional. Role-based access mostly correct with minor permission issues.	Authentication present but role-based access partially implemented or inconsistent.	Authentication missing or non-functional. No role-based access control.
Backend Additional Requirements (3%)	Content negotiation, seed script, API tests and Render deployment all fully functional.	Most requirements met. Minor issues with tests or deployment configuration.	Some requirements missing or partially implemented (e.g., limited tests, seed script incomplete).	Multiple requirements missing or non-functional.
Frontend Integration and Features (4%)	Frontend consumes backend API correctly. All CRUD operations work based on role. Bootstrap styling applied consistently. Authentication with secure token handling.	Frontend mostly functional. Minor issues with API integration or role-based rendering. Good styling applied.	Frontend partially functional. Some CRUD operations work. Basic styling present. Token handling implemented.	Frontend non-functional or major features missing. Poor integration with backend.

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Frontend Data Display and Forms (2%)	Data display with filtering, sorting and paging fully functional. Forms with comprehensive validation for Create and Update operations.	Data display mostly functional. Forms have good validation with minor gaps.	Basic data display. Forms present but validation incomplete or inconsistent.	Data display or forms non-functional. Little to no validation.
Frontend Testing and Deployment (2%)	Ten end-to-end tests covering main features, all passing. Successfully deployed to Vercel and accessible.	8-10 tests present, most passing. Deployed to Vercel with minor issues.	5-7 tests present. Deployment completed but may have accessibility issues.	Fewer than 5 tests or non-functional. Not deployed or deployment non-functional.

Code Quality and Best Practices - Learning Outcome 1 (40%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Code Readability and Maintainability (10%)	Code is exceptionally clean and readable with consistent formatting. Meaningful variable and function names. Well-organised file structure. Comprehensive comments where needed.	Code is clean and readable with good naming conventions. Organised structure with some helpful comments. Minor inconsistencies.	Code is mostly readable. Some unclear naming or inconsistent formatting. Basic organisation present.	Code is difficult to read with poor naming conventions. Inconsistent or chaotic structure.
Modularity and Reusability (10%)	Excellent use of modular, reusable components. Functions and components are well-abstracted. No code duplication. DRY principles followed throughout.	Good modular structure with reusable components. Minimal code duplication. Most code follows DRY principles.	Some modular components present. Some code duplication exists. Partial adherence to DRY principles.	Little modularity. Significant code duplication. Poor component abstraction.
Code Simplicity and Clarity (10%)	Code is simple, elegant and easy to understand. Complex logic is well-explained. Appropriate use of language features without over-engineering.	Code is generally simple and clear. Most logic is easy to follow with minor complex sections.	Code works but contains unnecessarily complex sections. Some logic is difficult to follow.	Code is overly complex or convoluted. Difficult to understand the implementation.
Performance and Resource Management (10%)	Excellent resource management. Efficient database queries. Proper error handling and memory management. No performance bottlenecks.	Good resource management. Mostly efficient queries. Adequate error handling with room for improvement.	Basic resource management. Some inefficient queries or error handling gaps. Performance acceptable but not optimised.	Poor resource management. Inefficient queries. Missing error handling. Performance issues present.

Version Control - Learning Outcome 1 (5%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Commit History and Messages (5%)	Regular, meaningful commits throughout development. Clear, descriptive commit messages. Logical progression showing development process. Clean commit history.	Regular commits with mostly clear messages. Commit history shows good development progression with minor gaps.	Commits present but irregular or clustered. Some commit messages unclear. Basic development history visible.	Few commits or poor timing. Unclear messages (e.g., ”update”, ”fix”). No clear development progression.

Reflection - Learning Outcome 1 (5%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Reflection Quality (5%)	Thoughtful reflection (300-500 words) covering all required points. Clear articulation of challenges, solutions and learning. Demonstrates deep understanding and self-awareness.	Good reflection covering all required points. Adequate discussion of challenges and learning with some depth. Within word count.	Reflection present covering most points. Brief or superficial discussion. May be slightly outside word count.	Reflection missing, incomplete or significantly outside word count. Lacks depth or doesn’t cover required points.

Presentation - Learning Outcome 1 (5%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Video Demonstration (5%)	Clear 5-10 minute recording demonstrating all backend and frontend features. Well-paced, professional presentation. Easy to follow with good audio/visual quality. Link submitted correctly.	Good recording within time limit. Most features demonstrated. Generally clear with minor audio/visual issues. Link submitted.	Recording present but may be too short/long. Some features demonstrated. Quality acceptable but could be clearer.	Recording missing, significantly outside time limit, unclear, or doesn’t demonstrate key features. Poor quality.